NSDI ’26 Fall Paper #656 Reviews and Comments
==========
Paper #656 Aether: An Efficient Incremental DNS Configuration Verification Framework

Review #656A
==========

Overall merit
---
3. Weak accept (A paper that you are okay with being accepted, you will not argue against it. The paper is correct with potentially limited novelty or unconvincing evaluation.)

Reviewer expertise
---
2. I have passing familiarity with this area

Experimental methodology
---
3. Average

Writing quality
---
3. Adequate

Paper summary

This paper presents Aether, a DNS configuration verification framework developed to address the inefficiencies and limitations identified in existing tools such as GRooT and Octopus. Aether is based on the approach that nameserver behavior can be represented as a match-action table, allowing for efficient and incremental DNS configuration verification. The framework features three main components:

1. A data structure designed to encode query spaces using Local Equivalence Classes (LECs), grouping DNS queries with identical behaviors to reduce the number of equivalence classes and enhance efficiency.

2. A symbolic execution algorithm developed to comprehensively cover all possible query spaces without executing actual queries.

3. An incremental verification mechanism that updates only the portions of the configuration that are affected by changes, eliminating the need for complete re-verification.

The evaluation of Aether uses real-world datasets and reports improvements over GRooT in various performance metrics, including up to 255.7 times faster end-to-end verification, a 6046-fold reduction in equivalence classes, and a 3370-fold speedup in incremental updates. Furthermore, Aether detects more DNS configuration errors than GRooT, indicating its capability and accuracy.

Strengths
---

1. Innovative Approach: Employing match-action tables and LECs to model nameserver behaviour represents a novel and effective methodology, facilitating substantial compression of equivalence classes and enhancing verification efficiency.

2. Incremental Verification: Aether's capability to process frequent DNS configuration updates incrementally marks a significant improvement over existing tools such as GRooT, which necessitate full re-verification.

3. Comprehensive Evaluation: The paper presents extensive experimental findings, benchmarking Aether against GRooT across multiple metrics—including LEC construction time, symbolic execution time, and end-to-end verification time. These results are persuasive and underscore Aether's superior performance.

4. Real-World Applicability: Analyses performed on real-world datasets, including those from university and census sources, demonstrate Aether's practical value in large-scale DNS environments.

5. Error Detection Capability: Aether identifies a broader range of DNS configuration errors than GRooT, notably detecting delegation inconsistencies that GRooT cannot identify due to implementation constraints.

Weaknesses
---

1. Complexity of Implementation: Although the hierarchical storage structure and symbolic execution algorithm are effective, their implementation and ongoing maintenance in real-world systems may pose significant challenges. The paper does not address the possible overhead associated with integrating Aether into existing DNS workflows.

2. Absence of Error Repair Mechanism: While Aether demonstrates strong capabilities in error detection, it lacks features for diagnosing or repairing erroneous configurations, which could further improve its practical value.

3. Clarity of Presentation: Certain sections, especially those detailing the symbolic execution algorithm and LEC construction, contain complex explanations that may be difficult to understand.

Comments for authors

Thank you for submitting your paper to the NSDI 2026 Fall deadline. The paper discusses advancements in DNS configuration verification using Aether, specifically addressing limitations found in existing tools such as GRooT. It describes the application of match-action tables, LECs, and incremental verification mechanisms which contribute to improvements in performance and error detection. The paper may benefit from including a broader comparison with other tools, an analysis of scalability, and clearer explanations of technical concepts. Addressing these points could strengthen the paper and clarify its contributions.

1. Implementation overhead: The hierarchical storage structure and symbolic execution algorithm, though instrumental to Aether's performance, introduce potential computational and storage costs. A thorough discussion quantifying these overheads is warranted, including benchmarks on resource utilization and maintenance complexity. Additionally, providing clear guidelines and best practices for integrating Aether within existing DNS workflows will assist practitioners in evaluating feasibility and adoption paths. This should encompass considerations for legacy system compatibility, incremental deployment strategies, and monitoring of operational impacts.

2. Error diagnosis and repair: Beyond error detection, extending Aether to support diagnostic and automated repair mechanisms would significantly enhance its practical value for DNS operators. Implementing features that categorize detected errors, suggest root causes, and offer actionable remediation steps could streamline operational workflows and reduce downtime. The development of a feedback system for continuous

improvement—where repaired configurations are re-verified—would further promote reliability and trust in the tool.

3. Paper writing: Certain sections of the paper, notably those detailing LEC construction and symbolic execution, are dense and may impede comprehension for a broader technical audience. It is advised to simplify these explanations through the inclusion of conceptual diagrams, step-by-step illustrative examples, and a high-level workflow overview preceding technical details.

4. Security considerations: The deployment of Aether necessitates a discussion on the security implications associated with verifying DNS configurations, particularly regarding the handling of sensitive data. Risks such as inadvertent exposure of confidential network information during verification processes should be addressed. Recommended mitigation strategies include data anonymization, access control policies, and secure handling protocols. Articulating these considerations underscores a commitment to responsible research and fosters confidence among stakeholders in adopting Aether for critical infrastructure verification.

5. Scalability analysis: While current results demonstrate Aether's effectiveness on sizable datasets, the scalability of the system for global-scale DNS infrastructures remains less explored. It is recommended to conduct experiments on datasets representative of a very large number of DNS records, reflecting real-world demands. This analysis should identify performance bottlenecks such as memory consumption, query latency, and synchronization overhead.

Review #656B

==========

Overall merit

---

1. Reject (A paper that must not be accepted -- you will fight against this
   paper. There are clear technical correctness issues and ostensibly no
   novelty.)

Reviewer expertise

---

3. I know the material, but am not an expert

Experimental methodology
---
3. Average

Writing quality
---
2. Needs improvement

Paper summary

This paper presents Aether, a verifier for DNS configuration. It first argues that the definitions of Equivalence Class (EC) in previous work GRoot is unsound, and inefficient. Then, the paper presents a new approach to compute ECs.

Strengths

- proposes a new method to verify DNS configurations

Weaknesses

- The novelty of the equivalence class definition is limited
- Lack clarify on why the proposed method works better than existing ones
- The writing needs significant improvement

Comments for authors

Thanks for submitting your work to NSDI. This paper studies an important problem of DNS verification. I appreciate the adaption of equivalence classes used by data plane verifiers to the verification of DNS. The results are promising in the total number of equivalence classes. However, there are still a lot of room for improvement, as detailed below.

The novelty of this paper is not clear, according to the current writeup.

First, the key idea of this paper is a method to compute equivalence classes. However, it seems just a simple adaption of equivalence classes from data plane verifiers like AP Verifier. The incremental update method is somewhat straightforward. More importantly, in the overview, the paper only shows how to compute ECs using Aether, without comparing with GRoot. So, it is not clear how the EC computation method improves over that of GRoot.

Second, the paper says the ECs computed by GRoot is unsound. While it is based on the results from the other paper. This paper should give more details on why this unsoundness happens, and why the approach presented in this paper is sound with some proof.

There are many places that need to be clarified.

In the Overview section, the paper says "GROOT does not have a mechanism to make such a determination, so it is unable to detect the occurrence of loops in a timely manner." This is confusing, since in the GRoot paper, it clearly defines the "Rewrite Loop" as one of the bugs that it can find. So why GRoot cannot find such bugs?

In the Introduction, the paper says "Octopus [38] plans to support incremental verification, but it has not yet been implemented." However, "not been implemented" should not be listed as a limitation. It would be better to differentiate Aether from [38] on the level of method or design.

In the Dataset description, the paper says the DNS information is from a university, but later on the paper says "The university dataset obtained from the laboratory", so is it from the university or lab?

In Experiment 1, the paper says "This demonstrates that Aether is highly effective in compressing equivalence classes. " However, it is not clear, even after reading the overview, why Aether can reduce the number of ECs.

Minors:

In Section 2, there is a fragment sentence " And the".
On Page 11, "zonefiles.we run the source code ". "GROOT found 3 number of hop errors."
Fonts of Figure 3 are too small.

Review #656C

==========

Overall merit

---

2. Weak reject (A paper that you want to reject, but will not champion

rejection if there are positive reviews (e.g., '3's or above). The paper
is correct with potentially limited novelty or unconvincing evaluation.)

Reviewer expertise
---
2. I have passing familiarity with this area

Experimental methodology
---
3. Average

Writing quality
---
2. Needs improvement

Paper summary

This paper describes algorithms and data structures for verifying DNS configurations. The authors introduce new data structures for storing equivalence classes of DNS queries local to each name server and a symbolic execution algorithm. The authors also give an algorithm for incremental analysis of changes to DNS configuraitons.

The evaluation demonstrates that the authors' new tool is considerably faster (by orders of magnitude) than Groot, an earlier DNS resolution tool.

Strengths

- important topic
- new data structures and compression algorithms for storing and determining equivalence classes of DNS queries
- incremental computing seems particularly important
- impressive performance numbers

Weaknesses

- unclear explanations and key typos in places

- insufficient discussion of related work, especially

[23] Si Liu, Huayi Duan, Lukas Heimes, Marco Bearzi, Jodok Vieli, David Basin, and Adrian Perrig. A formal framework for end-to-end dns resolution. In SIGCOMM'23, pages ACM, 932–949, 2023.

referenced in the paper. While there are many references to related work throughout the paper, there is no dedicated related work section to summarize the relationships between current and past work

- the paper would be stronger if there was a formal semantic analysis of the algorithms used, as in past papers on Groot and in work by Liu [23]

- limited discussion of incremental algorithms and their correctness

Comments for authors

Overall

This paper investigates efficient algorithms for DNS verification. It does so in a way inspired by Groot, but fixes some problems with Groot and improves the performance of the tool substantially, using a new data structure and by investigating incremental evaluation schemes. This reviewer found a number aspects of this paper difficult to understand --- deeper, clearer explanations would have been most helpful several places including:

- the essential modeling differences between Groot and this work here (this reviewer not being familiar with Groot). If a key contribution of this work is in fixing Groot's model so it is sound, we need to understand the fundamental problem with Groot and how it is resolved.

- how does the model in this paper relate to the model developed in reference [23] (Liu et al, SIGCOMM 23)? There is no discussion.

- What is a log exactly? Ie: L1, L2, etc. Is a log just a symbol associated with a zone file? Does it contain any additional information? Why exactly is the trace important, as opposed to the final answer received -- is it important just to find loops?

- The selective incremental reexecution of zone files was discussed quite tersely --- this would seem to be an important new algorithm. Its correctness is unclear

In addition, some past papers have provided semantic models of DNS and proofs of correctness. While such models may not be necessary here to justify the proposed algorithms, they would strengthen the work.

On the plus side, the performance numbers are very good, showing clear improvement in processing times over past work. Hence, the algorithms and data structures proposed here are important.

More comments

In the introduction, the authors write:

"1,000,000 zones, it takes GROOT about 8455.13s to finish the verification. These findings indicate that GROOT demonstrates inefficiency even with moderate dataset sizes."

It would be useful to give some context at this point. How does 1,000,000 zones compare with the size of large DNS systems? How does GROOT scale (ie, linearly, non-linearly)? How often does verification need to be done -- how slow 8400 seconds relative to the rate at which the process needs to occur in practice?

Also in the introduction, the authors observe "As mentioned in the prior study [23] the same EC can exhibit different resolution behaviors in GROOT. This inconsistency arises when the same type of query leads to one being resolved with an immediate detection of a rewrite loop, while another requires additional queries to identify the same loop." Also, several other places in the paper, the authors mention this issue with Groot. While it is helpful that the authors point out there is a problem with Groot, it would have been more helpful to explain the key modeling error and how the author's model differs from the Groot model and hence solves the problem. Though the authors come back to this problem several times the paper, it was unclear to this reviewer whether the problem was just an implementation bug that is easily fixed or if it is a deeper error of some kind that is not easily fixed. In section 6, for example, the authors write:

"Analyzing Groot's code and design, we discovered discrepancies between the implementation and the documentation. This led to an incomplete detection of the Delegation Inconsistency error."

Such language suggests that Groot's intended model may be ok but that the implementation is broken because it deviates from that model. More explanation is needed.

Glancing quickly at reference [23], it would seem the problems with Groot's semantics are deeper. However, in this case, it would have been helpful to point out the delta between this work and reference 23, but this paper contains little explanation.

The authors suggest a key insight in this work is that "Nameserver can be modeled as a match-action table." Already in the Groot paper, the authors there explain in section 4.1 that "The idea with our approach is that, instead of enumerating all possible queries, we can construct a collection of equivalence classes of queries ... Other verification tools such as Veriflow [29] and Atomic Predicates [46, 47] use a similar approach in the context of packet forwarding." In other words, the connection to packet forwarding and the data structures used there (such as match-action tables) was already known. Groot also uses symbolic execution. Hence, the key contribution would seem to be to take these ideas and refine them, creating better data structures and also pursuing incremental verification methods.

In 2.2.3, I was surprised to see that the authors in the "incremental verification" section, the authors write in the last paragraph: "Next, we regenerate all traces from scratch." This would not seem to be very incremental. I wonder if the authors intended to say something different?

typos

- sec 3.1: aciton --> action

- sec 3.1: "the current length of E" -- I think you mean the "number of elements" in E. I was not sure what "length" might refer to initially. (In hindsight, this is easy to understand but when trying to read the material for the first time, I was flummoxed.

- Algorithm 2, line 6:

zone_hit <- zonefile.origin, all_types, 1;

I think this was intended to be:

zone_hit <- GetSpace(zonefile.origin, all_types, 1);

This type of made it very hard for me to understand what was going on here. It took me a long time to figure out that that line might have been a typo and what the appropriate fix was.

- sec 3.3: "This record can lead to unlimited rewriting, where the domain name *.a.com is rewritten
as *.a.a.com, and then further rewritten as *.a.a.com, ad infinitum."

I think you meant (note: *.a.a.a.com vs *.a.a.com at the end): "This record can lead to unlimited rewriting, where the domain name *.a.com is rewritten
as *.a.a.com, and then further rewritten as *.a.a.a.com, ad infinitum."

Review #656D

==========

Overall merit

---

1. Reject (A paper that must not be accepted -- you will fight against this
   paper. There are clear technical correctness issues and ostensibly no
   novelty.)

Reviewer expertise

---

2. I have passing familiarity with this area

Experimental methodology

---

3. Average

Writing quality

---

2. Needs improvement

Paper summary

This paper proposes to model the zonefiles in each DNS name server as "match-action" tables, and uses that to compute equivalence classes (LEC), i.e., symolic representation of the set of queries that will be resolved

the same way by the zone file. The LECs can be used to create symbolic traces for resolving queries and checking properties. The paper describes the constructions of these equivalence classes, some optimizations, and some strategies for incremental verification.

Strengths

- DNS verification is an important problem.

Weaknesses

- This is submitted as an operational track paper (maybe by mistake?), but does not describe any actual deployment, operational experiments, or lessons learned.

- Qualitative difference from GRoot are not clear.

Comments for authors

DNS verification is an important problem, and the paper's overall approach is quite reasonable, i.e., modeling zone files as match-action tables with non-overlapping rules and leveraging the hierarchical structure of DNS.

The main problem is that this paper has been submitted to the operational track, but does not describe any actual deployment, operational experiments, or lessons learned. I wonder if this was done intentionally or by mistake, but this doesn't fit the expectation from an operational track paper.

The paper also seems to assume the reader is familiar with the details of GRoot, making it difficult to appreciate the paper's approach and how it is different and/or potentially improves on GRoot. Independent of the track a future version of this paper is submitted to, I suggest that the authors improve this part of the paper. Given that GRoot is their main "competitor", my suggestion is to spend some time describing GRoot's workflow and components to the point needed to appreciate the paper's contributions, and keep making comparisons more clearly throughout the paper using their running example.

Review #656E
==========

Overall merit
---
2. Weak reject (A paper that you want to reject, but will not champion rejection if there are positive reviews (e.g., '3's or above). The paper is correct with potentially limited novelty or unconvincing evaluation.)

Reviewer expertise
---
2. I have passing familiarity with this area

Experimental methodology
---
3. Average

Writing quality
---
3. Adequate

Paper summary

This paper proposes Aether, a DNS verification tool. Aether proposes a data structure for partitioning query space, a symbolic execution algorithm, and a a mechanism to support incremental computation. Experiments show the proposed approach acheives multiple orders of magnitude speedups as compared to other approaches.

Strengths

Improving DNS reliability is a very important problem. Paper is written clearly. Evaluation is extensive.

Weaknesses

The work only targets configuration correctness and it is unclear how much configuration errors affect DNS reliaibilty in practice. Prior work (Groot) addresses significant parts of this problem already and benefits over Groot are somewhat unclear.

Comments for authors

The problem is great. We need more works on solidifying DNS. This work is easy to motivate, as DNS is something of a central point of failure for the Internet and is error prone and is centralized etc. However, the core problems of DNS do not seem addressed here. DNS is regularly attacked, and those attacks proceed in certain ways. They infiltrate the software, etc.. This approach does nothing to address that. It would be helpful if more motivation could be provided as to whether configuration issues are a substantial cause of issues in the first place, as that question seems core to motivating this work.

Another thing is that DNS configurations are typically quite simple, at least much simpler than traditional configurations in the context of distributed routing. DNS setups are much more amenable to simpler techniques such as straightforward, replication of policies. The majority of DNS behavior is determined by the software or the inputs, neither of which are analyzed by the proposed approach. It would be helpful if more motivation could be provided as to why as to whether DNS configurations are actually a source of problems or not, and if so, what extent that is true.

"unsound definitions of equivalence class", "still suffer from some issues" - unclear what this means, can you clarify? In general there were many brief statements about other papers that almost seemed a bit like name-calling rather than saying anything of particular substance. It would be helpful if these statements could be restated into precise claims.

There is a key insight that a DNS server can be modeled as a match-action table. But most DNS errors don't come from these matchings. This model seems too simple to capture most errors.

" other DNS mechanisms, such as DNS caching and DNS Security Extensions (DNSSEC) [28], are excluded from consideration in this study to simplify the problem" - this seems like a quite big limitation. DNS caching is fundamental to the operations of DNS. It seems difficult to argue this approach is "correct" in any sense while not modeling such large portions of what DNS servers do.

There are many grammatical errors. That is no huge deal, you can remember to clean them up in the camera ready. I think running the text through a grammar checker should catch most of them.

"utilize multi-dimensional prefix Trie" - typo

The criticisms about Groot seem to come from its implementation rather than its design. If the implementation of Groot is lacking that is unfortunate. But that seems irrelevant to the scientific contribution of this paper.

Related to that, I looked at Groot and don't understand what aspects of its design are "unsound" or flawed or even inefficient in any way. Most of the "proof" seems to be from running a particular implementation of Groot rather than making observations about any part of Groot's design that seem to have shortcomings. One thing that may help is if you can provide some background on Groot in this paper along with an analysis of specifiically what aspects of the Groot design are flawed, what effects those flaws may have, and specifically what design decisions you made that overcomes them.

Conclusion section is a bit short. It may help to summarize the key conclusions of the paper here, remind the reader of your key contributions and evaluation results and findings etc.

More details on the experiments would be helpful. The platforms found errors for example -- were those errors injected by you or were they real configuration errors?

"As mentioned in the previous section, the definition of GRoot is unsound" - The previous section is section 5 which seems to not discuss GRoot at all. Or are you referring to a prevoius subsection or other text? If you do have some specific description about GRoot describing this "unsound"ness that would indeed be helpful. But I was not able to find it.

Some of the ideas in Section 3 seem interesting. It would probably help if in the intro or somehwere you could lay out more specifically what your contributions are and in general center the paper's writing around those. Most of what you mention in the introduction doesn't really seem novel, symbolic execution etc. The ways you apply it may be but it's hard to tell from the writing as the intro/related work/evaluation/etc seem centered around much more vague claims of "unsoundness".

Text is too small to read in figures (e.g., Figure 3).

I feel somewhat confused by the results of the evaluation in particular why you would have such a substantial performance improvement over GRoot. It may help if you could do some microbenchmarks to show which specific aspects of your design in Sections 3-5 lead to these benefits. One worry

I have is most of your benefits are simply coming from Section 5.2 (or even in general section 5) which is a relatively straightforward (right?) addition to your design and one that GRoot would have been fully compatible with as well, rather than any of your actual core contributions.

The key design contributions listed in the introduction do not seem novel. GRoot seems to do all of them already, symbolic execution, treating DNS like match-action table, etc.

There are all these mentions of unsoundness but GRoot's paper has a proof of correctness and your paper does not. Is there some part of their proof which you think is incorrect? It would be helpful if you could point specifically to where the issue is. Likewise, a proof of correctness of your work would be helpful (even a discussion of what correctness really means here -- are you claiming you can find all bugs of certain types, under certain circumstances, given certain inputs? What types/circumstances/inputs/etc?) Related to this, the paper may benefit from a crisp problem statement and problem formulation.

Review #656F

Overall merit

2. Weak reject (A paper that you want to reject, but will not champion rejection if there are positive reviews (e.g., '3's or above). The paper is correct with potentially limited novelty or unconvincing evaluation.)

Reviewer expertise

2. I have passing familiarity with this area

Experimental methodology

3. Average

Writing quality

2. Needs improvement

Paper summary

This paper presents Aether, a tool for verifying DNS configurations. The key idea is to model the behavior of DNS name servers as a match-action table, turning query handling into a structured matching process. Building on this abstraction, the authors design an optimized data structure and query algorithm, together with an incremental verification algorithm that efficiently reuses prior results when configurations change. Evaluation shows that Aether achieves more than a 3,000× speedup over the prior state-of-the-art verifier Groot.

Strengths

- The paper targets at an important research problem.
- The reported evaluation result is strong (3000x speedup compared to SOTA).

Weaknesses

- The novelty of proposed solutions feels incremental.
- The evaluation is narrow as it only evaluates on a single dataset.
- Not a suitable submission to the operational system track.

Comments for authors

Thank you for submitting this work to NSDI 2026. The paper targets an important and relevant problem, and I appreciate the effort the authors put into explaining the problem background and motivating the need for better verification support. Meanwhile, I do have several key concerns that are not well answered in the current version.

My main concern is that the paper does not clearly justify its technical challenges and the novelty of its approach. It is often unclear what is novel or unique about the proposed design compared to prior work, especially GRoot. As written, many parts of the system read like an optimized version of GRoot. This makes the overall contribution feel incremental. Related to this, I would like the authors to be much more explicit about how their system relates to GRoot. Did you build on top of GRoot's implementation, or is everything implemented from scratch? The paper itself notes that several design choices are "akin to" those in GRoot, but it does not clearly distinguish what is reused versus what is fundamentally different.

At the moment, the paper also lacks a proper related work section, which further exacerbates the scope issues. It is quite for readers to position this work correctly without an extensive discussion of how this work compares to other DNS verification literature and to the broader data-plane verification line of work.

Evaluation-wise, I find the results rather weak. The system is evaluated on essentially a single dataset, which raises questions about the generality of the findings. It is also unclear whether the proposed approach introduces false positives. Overall, the current evaluation does not fully convince me that the system would behave robustly in diverse real-world settings.

Additionally, I'm a bit unsure if the paper select its submission track correctly. The paper is marked as an "operational systems" submission, but the content does not really share any operational experience or deployment lessons from running the system in production. This paper looks more like a typical research prototype paper.

Comment @A1 by Reviewer A

Thank you for submitting to the USENIX NSDI 2026 Fall edition. Reviewers valued the problem formulation and evaluation, but the paper was not accepted due to insufficient emphasis on novelty, unclear writing, lack of operational experience, limited comparison to works like GRoot, and a narrow evaluation.